

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..4

CONFIGURATION

 Copy as Markdown

Configure email sign-in

This page covers configuration options for the email identity provider beyond the basic setup.

Configuration options

New projects use `ServerpodCloudEmailIdpConfig`. It reads the `pepper` from the `emailSecretHashPepper` password, but it does not take the other options on this page. To set them, switch to `EmailIdpConfigFromPasswords` and pass your own `sendRegistrationVerificationCode` and `sendPasswordResetVerificationCode` callbacks. See [Use your own email provider](#).

Below is a non-exhaustive list of some of the most common configuration options. For more details on all options, check the `EmailIdpConfig` in-code documentation.

Peppering

A pepper is a server-side secret mixed into a password before it is hashed and stored, so a database leak alone is not enough to crack the hashes. See [storing secrets](#) for where to keep it.

The pepper is configured via the `secretHashPepper` property in `EmailIdpConfig`, or read from the `emailSecretHashPepper` password by `EmailIdpConfigFromPasswords`, as described in the [server-side configuration](#) section. Its [recommended pepper length](#) is 32 bytes.

WARNING

If the pepper is changed, all passwords in the database will need to be re-hashed with the new pepper, or the old pepper needs to be added as a fallback pepper. Store the pepper securely and never expose it to the client.

Rotating secret hash peppers



Ask AI

To keep old passwords valid after a pepper change, use the `fallbackSecretHashPeppers` configuration option to add the old pepper as a fallback pepper.

```
final emailIdpConfig = EmailIdpConfigFromPasswords(
  fallbackSecretHashPeppers: [
    pod.getPassword('oldEmailSecretHashPepper')!,
  ],
);
```

Customizing password requirements

By default, the minimum password length is set to 8 characters. If you wish to modify this requirement, you can use the `passwordValidationFunction` configuration option.

```
final emailIdpConfig = EmailIdpConfigFromPasswords(
  passwordValidationFunction: (password) {
    // Require at least 12 characters, one uppercase, one lowercase, one number
    if (password.length < 12) return false;
    if (!password.contains(RegExp(r'[A-Z]'))) return false;
    if (!password.contains(RegExp(r'[a-z]'))) return false;
    if (!password.contains(RegExp(r'[0-9]'))) return false;
    return true;
  },
);
```

TIP

This is useful to ensure password policies on the server-side. It is a best practice to pair it with a configuration on the client-side to provide a better UX when creating a new password. The `EmailSignInWidget` and `EmailAuthController` have a `passwordRequirements` parameter that can be used to configure the password requirements.

Custom verification code generation

You can customize how verification codes are generated:

```
final emailIdpConfig = EmailIdpConfigFromPasswords(
  registrationVerificationCodeGenerator: () {
    // Generate a 6-digit numeric code
    final random = Random.secure();
    return List.generate(6, (_) => random.nextInt(10)).join();
  },
);
```

WARNING

Remember to configure the `verificationCodeConfig` parameter on the `EmailSignInWidget` to match the length and allowed characters of the verification code you generate. Otherwise, users will never be able to enter the verification code correctly. See the [customizing the UI](#) section for more details.

Bypassing verification code in development

To simplify testing, it is possible to bypass the verification code by using a custom verification code generator that always returns the same code. This can be configured only for the development run mode as below:

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    EmailIdpConfigFromPasswords(  
      registrationVerificationCodeGenerator: pod.runMode == 'development'  
        ? () => '11111111' // Digits only, and the same length as production.  
        : defaultVerificationCodeGenerator,  
    ),  
  ],  
);
```

Reacting to events

Beyond the `sendRegistrationVerificationCode` and `sendPasswordResetVerificationCode` callbacks to send verification codes to the user, there are a few callbacks that can be used to react to events, such as `onAfterAccountCreated` and `onPasswordResetCompleted`. These can be useful for sending emails to the user to communicate the successful operation.

Rate limiting

The email provider includes built-in rate limiting to prevent abuse, with some sensible defaults:

- **Failed login attempts:** Limited to 5 attempts per 5 minutes
- **Password reset attempts:** Limited to 3 attempts per hour
- **Verification code attempts:** Limited to 3 attempts per code lifetime of 15 minutes

You can customize all rate limits in the `EmailIdpConfig`:

```
final emailIdpConfig = EmailIdpConfigFromPasswords(  
  registrationVerificationCodeLifetime: Duration(minutes: 15),  
  registrationVerificationCodeAllowedAttempts: 3,  
  passwordResetVerificationCodeLifetime: Duration(minutes: 15),  
  passwordResetVerificationCodeAllowedAttempts: 3,  
  failedLoginRateLimit: RateLimit(  
    maxAttempts: 10,  
    timeframe: Duration(minutes: 10),
```

```
),  
maxPasswordResetAttempts: RateLimit(  
  maxAttempts: 5,  
  timeframe: Duration(hours: 2),  
),  
);
```